AI-Powered Restaurant Operations & Review Assistant

System architecture: Yelp Open Dataset to manager query response (FastAPI + LangGraph + Hybrid RAG + Streamlit)

A OFFLINE INGESTION & INDEXING

run once, before serving queries

1 Yelp Open Dataset

business.json + review.json (Kaggle)
Fields: business_id, name, categories, city, state, stars, review_count, attributes, text, date, useful

2 Load & preprocess

dataset_loader.py, preprocess.py
Keep restaurant businesses, join reviews to business metadata, clean text, write processed_reviews.csv

3 Complaint enrichment

enrichment_service.py (rule-based)
8 categories, sentiment, aspect, severity 1-5, priority score 1-100, is_complaint flag

4 SQLite store

enriched_reviews: 48,288 reviews, 3,934 restaurants
Also: users, user_feedbacks, chat_history

5 Document builder

No chunking: 1 review = 1 document
Metadata per doc: restaurant, city, state, rating, category, severity, priority, date

6 Embeddings + ChromaDB

all-MiniLM-L6-v2 (384-d, cosine)
Indexed: 500 prioritised reviews
(build_vector_index.py)

7 BM25 keyword index

rank-bm25 over review text
Tokenised on non-alphanumerics, lower-cased

fallback + names

vector + keyword indexes

B ONLINE QUERY PIPELINE

per manager query

Restaurant manager (UI)

Streamlit app (frontend/app.py)
Login, natural-language query box, filters (restaurant, category, city, rating), complaint dashboards, chat history, admin console
Via api_client.py over HTTP

FastAPI service /api/v1

~30 endpoints incl.
/query /search /analyze /prioritize /agent/workflow /forecast /similar-restaurants /customer-satisfaction /feedback /chat/history /admin/*
Pydantic schemas, CORS, OpenAPI docs at /docs

Input guardrails

guardrails_service.py
- Empty / length check
- Prompt-injection regexes
- Out-of-scope patterns (code, crypto, medical, trivia)
- Restaurant domain-keyword check

Fail: safe fallback reply, no retrieval

valid

Hybrid retriever

hybrid_search.py
- Dense top-30 (ChromaDB) + BM25 top-30
- Reciprocal Rank Fusion, k=60, weights 0.5 / 0.5
- Restaurant-name detection
- Metadata filters: restaurant, category, city, rating
- SQLite fallback if too few hits

Output: top-k reviews + scores

RAG service

rag_service.py, operations_analyst_service.py
- Assemble cited context from retrieved reviews
- Gemini 2.5 Flash / Pro via OpenRouter
- Deterministic grounded fallback if no API key

Output: grounded answer with evidence

LangGraph multi-agent

1 Classifier
category, severity, priority tier

2 Root cause
causes + evidence snippets

3 Recommendation
SOPs, steps, guest recovery

4 Business impact
churn risk, revenue exposure

5 Quality judge
rubric scores + compliance flags

dashed: retry if judge score < 0.70 (planned)

Decision package

Returned to the UI:
- Similar historical reviews and cases
- Complaint category, severity, priority rank
- Root causes with evidence
- Recommended corrective actions (SOP, timeline)
- Churn / revenue risk
- Quality verdict + human-review flag

analytics calls

log history

scored by

feedback / admin

C SUPPORTING SERVICES, STORAGE & EVALUATION

called by the API alongside the main pipeline

Analytics services (SQLite enriched_reviews)

Prioritization
priority = severity, frequency, recency; ranks complaint categories

Satisfaction risk
explainable 0-100 score from ratings, complaint rate, severity; churn tier

Root-cause analysis
category to cause mapping with evidence reviews; LLM-assisted when key present

Similar restaurants
finds restaurants with similar complaint profiles for comparison

Operations analyst
orchestrates retrieval, analysis and answer for complex manager questions

Dashboard analytics
network stats, category mix, rating distribution

Trend forecasting

prophet_forecast.py
Facebook Prophet on weekly / monthly complaint and review volume per category or restaurant
Fallback: simple statistical projection when data is sparse
Endpoint: /forecast

Persistence & feedback

SQLite tables: users, chat_history, user_feedbacks
RAG answers and agent runs are logged per user
Feedback endpoint stores thumbs / comments for review in the admin console
(Feedback is stored; not yet fed back into ranking - planned)

Evaluation

evaluator.py, benchmark_dataset.py
Benchmark queries with expected categories
RAG quality metrics: faithfulness, answer relevancy, context precision / recall
BM25 vs vector vs hybrid comparison
Report generator: Evaluation_Report.pdf
(DeepEval G-Eval + LLM judge: planned upgrade)

Admin console

User and role management
Review CRUD
Feedback moderation
Audit logs and system metrics
Roles: admin, operations_analyst, store_manager, user
(Token auth + hashed passwords: planned)

Legend

Data ingestion / preparation
Application & API services
Retrieval, LLM & agents
Storage / indexes
Security guardrails
Evaluation & admin
Planned upgrade (not yet implemented)
Data / control flow

External: OpenRouter API (Google Gemini 2.5 Flash / Pro) for generation. Local: sentence-transformers MiniLM embeddings, ChromaDB, SQLite, rank-bm25, Prophet. Frameworks: FastAPI, LangGraph, Streamlit.